SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:40**

Annexure A

Industry Problem Solving Task

Code of Conduct:

*I **N. Murali sai (192372309)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Smart Banking and Financial Management System

(a) Contribution of OO Principles, Layered Architecture & Design Patterns

The Smart Banking and Financial Management System is built on four object-oriented pillars — encapsulation, abstraction, inheritance and polymorphism — combined with a layered architecture and the Factory, Strategy, Observer and Repository patterns. Together these choices give the system the scalability, security, flexibility and maintainability that a financial institution requires.

- **Encapsulation & Security:** Account balances, PINs and transaction details are hidden behind well-defined class interfaces (e.g. Account.withdraw(), Account.getBalance()), so sensitive banking data can only be changed through validated, audited methods rather than direct field access.
- **Abstraction:** Abstract classes/interfaces such as Account and PaymentMethod expose only the operations that callers need (deposit, transfer, payLoanInstallment), hiding internal computation such as interest accrual or fraud-scoring logic.
- **Inheritance & Polymorphism:** SavingsAccount, CurrentAccount and LoanAccount extend a common Account base class; the service layer can treat any account polymorphically, so adding a new account type (e.g. a digital-wallet account) requires no change to existing transfer or statement code — this is the Open/Closed Principle in action.
- **Factory Pattern:** An AccountFactory (and similarly a LoanFactory) centralises the logic for instantiating SavingsAccount, CurrentAccount or LoanAccount objects. New account products can be introduced by adding a new concrete class and a factory branch, without touching client code — this directly improves flexibility and scalability.
- **Strategy Pattern:** Loan-interest calculation (flat rate, reducing balance, floating rate) and payment-processing policies are encapsulated as interchangeable InterestStrategy / PaymentStrategy objects. The bank can plug in a new interest scheme at runtime, satisfying regulatory or product changes without redeploying the whole loan module.
- **Observer Pattern:** TransactionObserver and FraudAlertObserver objects subscribe to Account/Transaction events. When a suspicious transaction occurs, the subject notifies all registered observers (SMS service, fraud-monitoring dashboard, compliance log) automatically — decoupling the core banking logic from notification channels and strengthening security through real-time alerting.
- **Repository Pattern:** An AccountRepository / TransactionRepository hides the OODBMS access details behind a simple, collection-like interface (save(), findById(), findByCustomer()), so the domain layer never depends directly on database APIs — improving maintainability and testability.
- **Layered Architecture:** Separating presentation, service, domain and data-access layers means the UI can change (web, mobile, ATM) without touching business rules, and business rules can change without touching persistence — each layer has a single responsibility, which is the core of maintainability and scalability under growing transaction volumes.

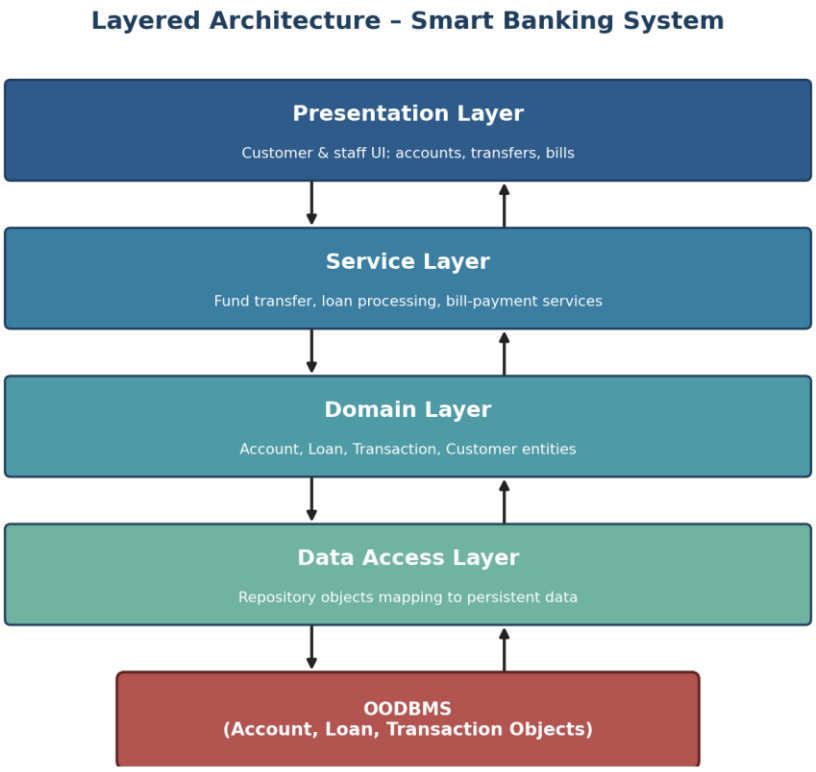

Figure 1.1 — Layered architecture and OODBMS interaction for the Smart Banking and Financial Management System.

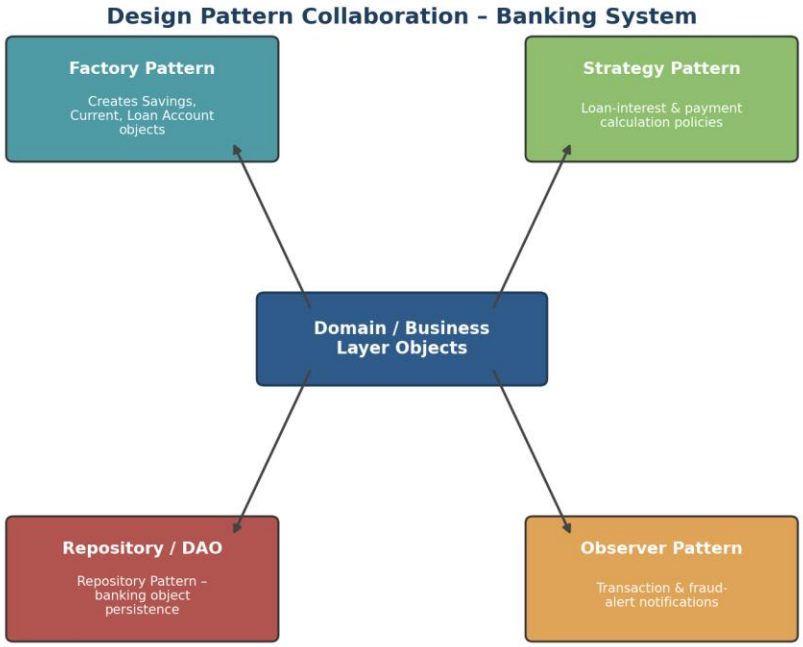

Figure 1.2 — Collaboration between the Factory, Strategy, Observer and Repository/DAO patterns and the domain layer.

(b) Layer Communication with the OODBMS

Communication with the OODBMS follows a strict top-down/bottom-up flow through the layers. A request from the presentation layer (e.g. 'transfer funds') is passed to the service layer, which orchestrates the use case by invoking

domain objects (Account, Transaction) that enforce business rules such as sufficient-balance checks. When persistence is required, the domain/service layer calls the data-access layer's repository interface rather than the database directly. The Repository/DAO implementation translates domain objects into OODBMS object references, using the database's object handles/OIDs to store and retrieve Account, Loan and Transaction objects directly as objects — preserving class identity, relationships (e.g. Customer holds a collection of Account references) and inheritance hierarchies without the object-relational mapping overhead a relational database would need. Read operations follow the reverse path: the OODBMS returns persistent objects, the repository wraps them back into domain objects, and the service layer forwards the result upward to the presentation layer. This clean separation means the OODBMS can be scaled, clustered or replaced with minimal impact on the upper layers, since only the data-access layer is aware of it.

Question 2: Smart Retail and Inventory Management System

(a) Contribution of OO Principles, Layered Architecture & Design Patterns

The Smart Retail and Inventory Management System applies object-oriented principles, a four-layer architecture and the MVC, Factory, Observer and DAO patterns to keep the system flexible as product ranges, suppliers and sales channels grow.

- **Encapsulation:** Product, Stock and Order classes encapsulate quantity, price and reorder-level fields, exposing controlled methods (Stock.reduce(), Stock.replenish()) so inventory counts can never be corrupted by direct external manipulation.
- **Abstraction & Inheritance:** A base Product class is specialised into ElectronicsProduct, GroceryProduct, ApparelProduct, etc., each overriding category-specific behaviour (e.g. expiry handling for groceries) while the sales and ordering logic works against the abstract Product type — new categories plug in without modifying existing code.
- **MVC Pattern:** The presentation layer separates Model (Product/Order data), View (staff dashboards, POS screens) and Controller (handles user actions), so the UI can be redesigned or a new channel (mobile POS) added without touching business logic, improving flexibility.
- **Factory Pattern:** A ProductFactory creates the correct concrete product subclass based on category codes, centralising object-creation logic so the business layer stays decoupled from concrete classes — improving scalability as new product lines are introduced.
- **Observer Pattern:** Stock objects act as subjects that notify registered LowStockObserver/managers when quantity falls below a minimum threshold, enabling automatic reorder alerts without embedding notification code inside the core stock class — this decoupling is central to maintainability.
- **DAO Pattern:** ProductDAO, OrderDAO and StockDAO isolate all persistence calls, so switching or upgrading the OODBMS, or adding caching, does not require changes to business logic — supporting long-term maintainability.
- **Layered Architecture:** Presentation, business, domain and data-access layers each evolve independently: warehouse-automation or demand-prediction features can be added to the business layer later without disturbing the UI or persistence layers, giving the system room to scale.

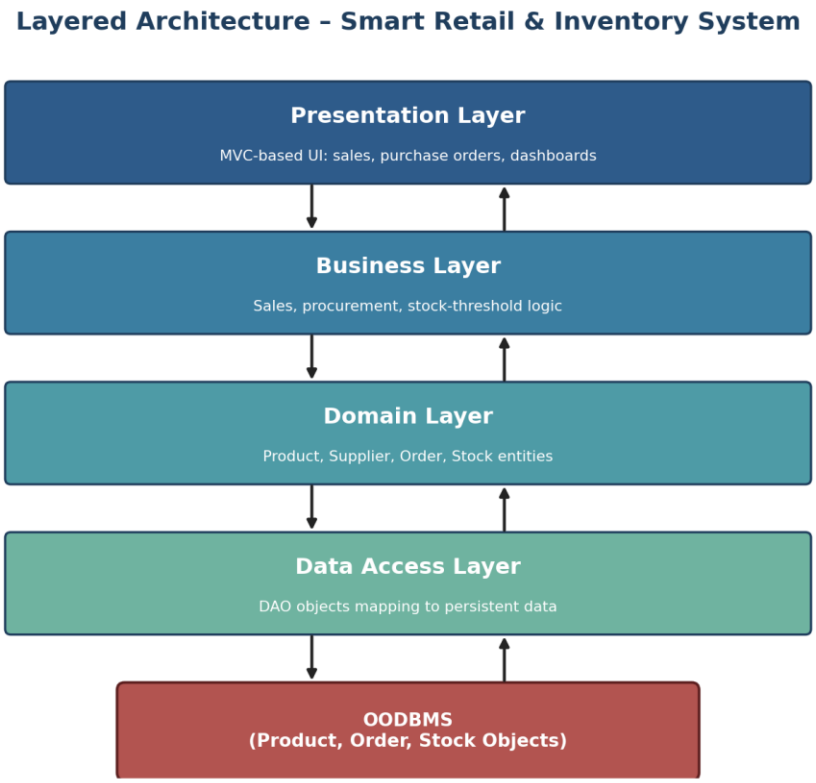

Figure 2.1 — Layered architecture and OODBMS interaction for the Smart Retail and Inventory Management System.

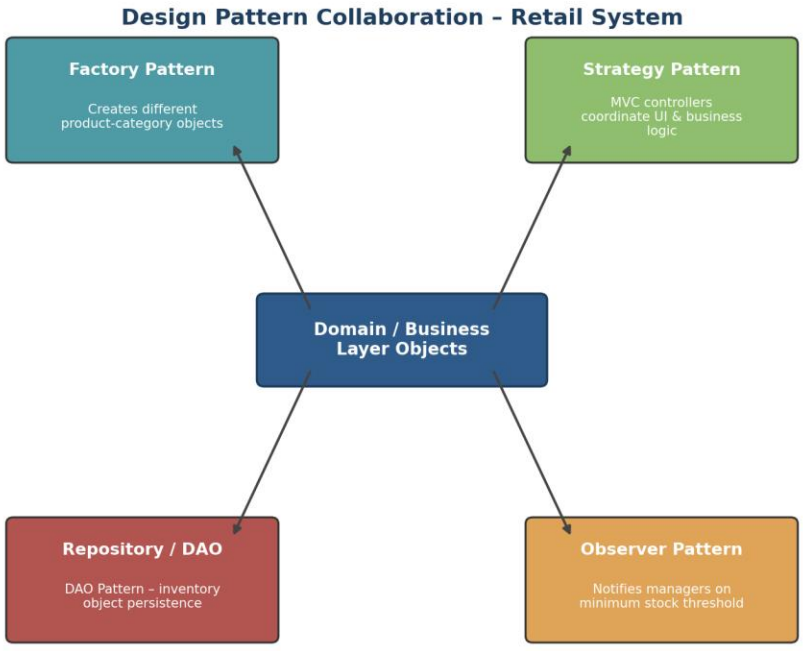

Figure 2.2 — Collaboration between the Factory, Strategy, Observer and Repository/DAO patterns and the domain layer.

(b) Layer Communication with the OODBMS

When a sale or stock update occurs, the presentation layer's controller passes the request to the business layer, which applies rules (e.g. checking stock availability, applying supplier constraints) using domain objects such as Product,

Order and Stock. To persist or retrieve these objects, the business/domain layer calls the corresponding DAO in the data-access layer rather than issuing database calls itself. The DAO uses the OODBMS's native object-persistence API to store Product, Supplier, Order and Stock objects directly as objects, preserving their class hierarchy (e.g. a query for all Product objects transparently returns the correct ElectronicsProduct or GroceryProduct subtype) and their relationships (an Order holding a collection of OrderLine objects, each referencing a Product). On retrieval, the OODBMS returns live object graphs which the DAO hands back to the business layer, which in turn formats results for the presentation layer. Because only the DAO layer talks to the OODBMS, the system can migrate to a distributed or replicated OODBMS cluster to handle peak sales traffic without any change to the business or presentation layers.

Question 3: Smart Hotel and Hospitality Management System

(a) Contribution of OO Principles, Layered Architecture & Design Patterns

The Smart Hotel Management System combines core OO principles with a layered architecture and the Strategy, Factory, Observer and Repository patterns to support flexible pricing, varied room/service types and reliable guest communication.

- **Encapsulation:** Reservation and Room classes protect booking status and availability behind methods such as Reservation.confirm() and Room.checkIn(), preventing invalid state changes (e.g. double-booking) from outside the class.
- **Inheritance & Polymorphism:** A base Room class is extended by StandardRoom, DeluxeRoom and SuiteRoom, and a base Service class by RoomService, SpaService, LaundryService; the reservation engine manipulates all of them through the common base type, so new room or service categories are added with zero change to booking logic.
- **Strategy Pattern:** Different PricingStrategy implementations (seasonal rate, loyalty discount, corporate rate) can be swapped at runtime for a given reservation, letting the hotel change pricing policy without modifying the reservation or billing classes — a direct flexibility gain.
- **Factory Pattern:** A RoomFactory / ServiceFactory creates the correct Room or Service subtype from a request, keeping object-creation logic out of the booking workflow and easing the addition of future offerings like smart-room packages.
- **Observer Pattern:** ReservationObserver and ServiceNotificationObserver objects are notified automatically when a reservation is created, modified or a service is requested, driving front-desk alerts and housekeeping notifications without coupling the core Reservation class to specific notification channels.
- **Repository Pattern:** GuestRepository, RoomRepository and ReservationRepository provide a uniform interface for persistence, so business components never issue direct OODBMS calls — this isolation is what keeps the system maintainable as new modules (guest feedback, loyalty programme) are added.
- **Layered Architecture:** Presentation, service, domain and data-access layers let the front-desk UI, business rules and persistence evolve independently, so features like smart-room automation can be added to the service/domain layers without rewriting the booking UI.

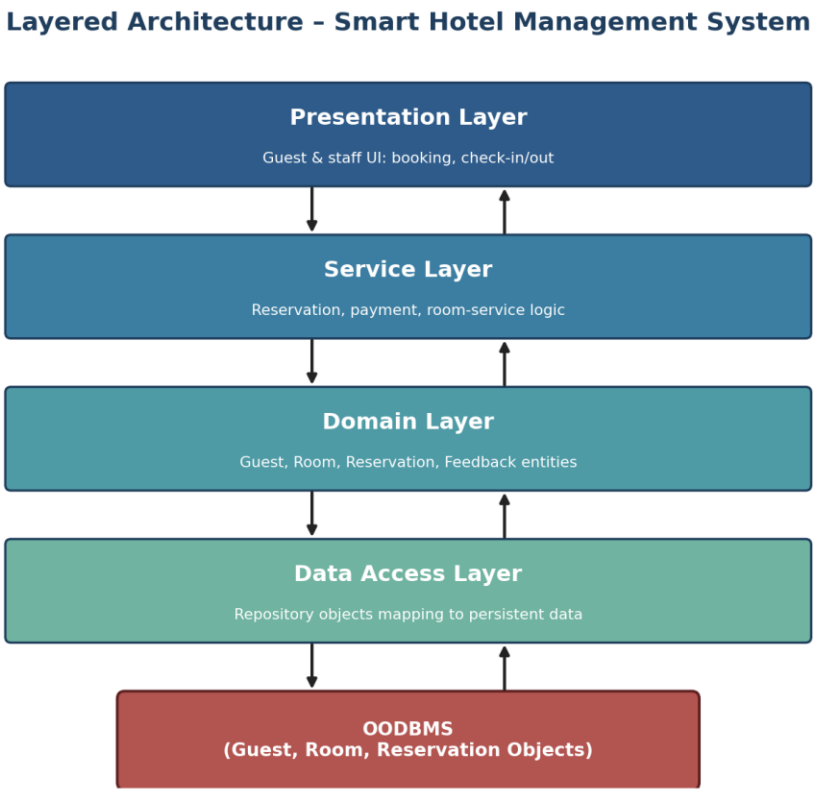

Figure 3.1 — Layered architecture and OODBMS interaction for the Smart Hotel and Hospitality Management System.

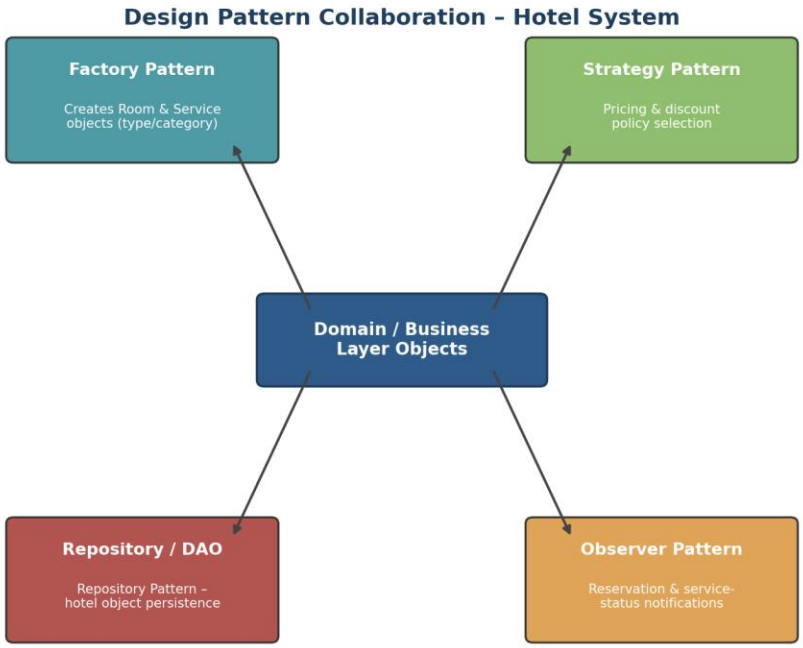

Figure 3.2 — Collaboration between the Factory, Strategy, Observer and Repository/DAO patterns and the domain layer.

(b) Layer Communication with the OODBMS

A booking request travels from the presentation layer to the service layer, which coordinates domain objects (Guest, Room, Reservation) to apply business rules such as availability checks and the selected pricing strategy. Persistence

is never handled directly by these domain objects; instead the service layer calls the Repository Pattern's interface in the data-access layer. The repository implementation converts domain objects into calls on the OODBMS, storing Guest, Room, Reservation and Feedback objects as native persistent objects, which keeps object identity and associations intact — for example, a Reservation object retains direct object references to its Guest and Room objects rather than needing foreign-key joins. When the front desk looks up a guest's history, the repository retrieves the persistent object graph from the OODBMS and hands reconstructed domain objects back up through the service layer to the presentation layer for display. Because the data-access layer is the sole point of contact with the OODBMS, the hotel chain can extend the schema (e.g. add smart-room automation objects) or scale the database across properties without impacting the reservation workflow or the guest-facing UI.

Question 4: Smart Logistics and Delivery Management System

(a) Contribution of OO Principles, Layered Architecture & Design Patterns

The Smart Logistics and Delivery Management System relies on object-oriented principles, layered architecture and the Strategy, Factory, Observer and DAO patterns to keep route planning, fleet management and tracking adaptable as delivery volumes and vehicle types grow.

- **Encapsulation:** Shipment and Vehicle classes encapsulate status, location and capacity fields, exposing controlled operations (Shipment.updateStatus(), Vehicle.assignRoute()) so state transitions stay consistent and auditable.
- **Inheritance & Polymorphism:** A base Shipment class is specialised into StandardShipment, ExpressShipment and FragileShipment, each with its own handling rules, while the dispatch engine works with the abstract Shipment type — new shipment categories (e.g. temperature-controlled) can be added without modifying dispatch logic.
- **Strategy Pattern:** Route-selection algorithms (shortest-distance, least-time, fuel-optimal) are implemented as interchangeable RouteStrategy objects, letting the system swap in a new optimisation algorithm — including a future AI-based one — without changing the dispatch or tracking classes.
- **Factory Pattern:** A ShipmentFactory creates the correct shipment subtype based on order attributes, centralising creation logic and making it easy to introduce new shipment types as the business expands into new delivery segments.
- **Observer Pattern:** DeliveryStatusObserver objects (customer notification service, dispatcher dashboard) subscribe to shipment status changes and are notified in real time when a shipment is picked up, in transit, or delivered, decoupling core tracking logic from the many notification channels involved.
- **DAO Pattern:** ShipmentDAO and VehicleDAO isolate persistence operations, so the OODBMS can be scaled or partitioned by region as delivery volume grows, without any change to the business layer that plans routes and assigns vehicles.
- **Layered Architecture:** Presentation, business, domain and data-access layers allow the tracking UI, routing algorithms and persistence to evolve at different speeds — for instance, autonomous-vehicle integration can be added to the domain layer without touching the customer-facing tracking screens.

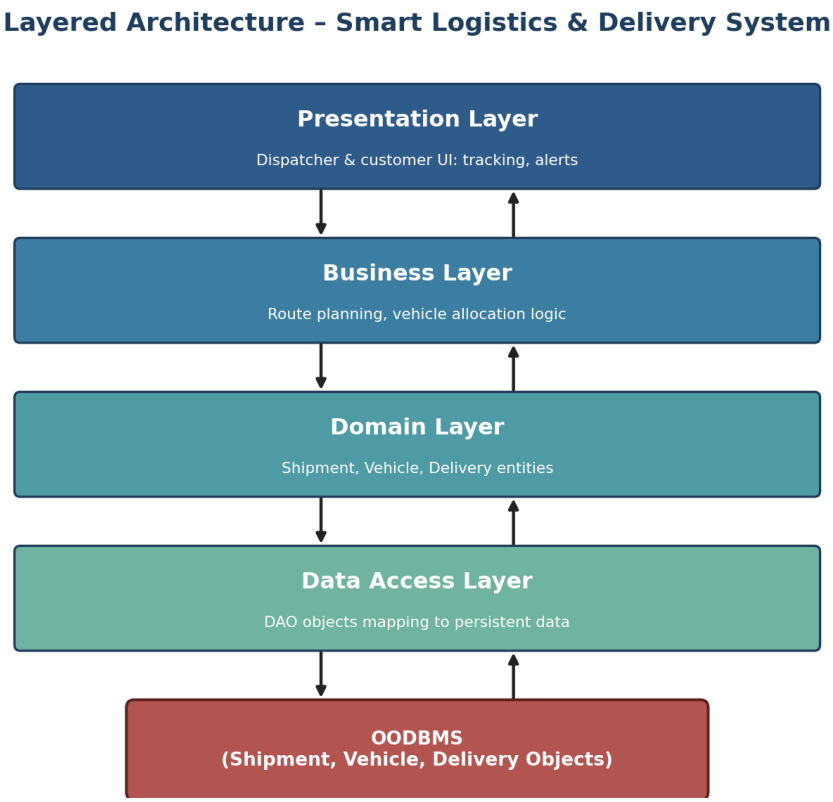

Figure 4.1 — Layered architecture and OODBMS interaction for the Smart Logistics and Delivery Management System.

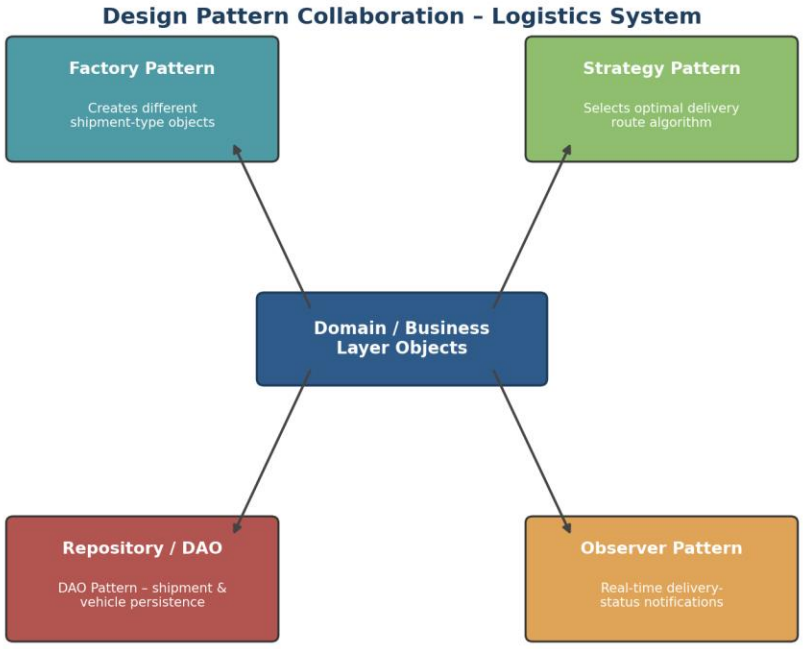

Figure 4.2 — Collaboration between the Factory, Strategy, Observer and Repository/DAO patterns and the domain layer.

(b) Layer Communication with the OODBMS

When a shipment is created or its status changes, the presentation layer forwards the request to the business layer, which uses domain objects (Shipment, Vehicle, Delivery) together with the selected RouteStrategy to decide on routing and allocation. Persistence needs are delegated to the DAO in the data-access layer rather than being handled inside the business logic. The DAO stores and retrieves Shipment, Vehicle and Delivery objects directly in the OODBMS as persistent objects, preserving object references such as a Delivery holding a live reference to its assigned Vehicle and Shipment objects, which avoids the relational joins that would otherwise be required to reconstruct these relationships. When the dispatcher or customer requests live tracking information, the DAO fetches the relevant persistent objects from the OODBMS and the business layer converts them into the response shown by the presentation layer. Because only the data-access layer interacts with the OODBMS, the logistics company can distribute the database across regional data centres to handle real-time tracking at scale without any change to the routing algorithms or the customer-facing tracking interface.

Question 5: Smart Energy Management System

(a) Contribution of OO Principles, Layered Architecture & Design Patterns

The Smart Energy Management System uses object-oriented principles, a layered architecture and the Observer, Strategy and Factory patterns, together with a Repository layer, to keep monitoring, analysis and alerting extensible as smart-meter numbers and analytics needs grow.

- **Encapsulation:** SmartMeter and Consumption classes encapsulate raw readings and computed usage figures behind methods such as Meter.recordReading() and Consumption.calculateUsage(), keeping measurement data consistent and protected from invalid updates.
- **Inheritance & Polymorphism:** A base SmartMeter class is extended into ResidentialMeter, CommercialMeter and, in future, RenewableSourceMeter, so the monitoring engine can process all meter types polymorphically and new meter categories can be added without touching existing analysis code.
- **Observer Pattern:** ConsumptionAlertObserver objects subscribe to Consumption objects and are notified automatically whenever usage crosses an abnormal threshold, decoupling the alerting mechanism (SMS, dashboard, email) from the core measurement logic — central to the system's extensibility.
- **Strategy Pattern:** Energy-optimization algorithms (peak-shaving, load-balancing, demand-response) are implemented as interchangeable OptimizationStrategy objects, so a new AI-based consumption-prediction algorithm can be introduced later simply by adding a new strategy class.
- **Factory Pattern:** A SmartMeterFactory creates the correct meter subtype from device registration data, keeping the domain layer free of meter-specific instantiation logic and easing future integration of renewable-energy meters.
- **Repository Pattern:** MeterRepository and ConsumptionRepository provide a uniform persistence interface, so the business and domain layers never depend on OODBMS-specific code — improving maintainability as the data volume from thousands of meters grows.
- **Layered Architecture:** Presentation (dashboards), business (analysis), domain (rules) and data-access layers are cleanly separated, so analytics capability can be extended in the business layer, or renewable-energy monitoring added to the domain layer, without disrupting the dashboards operators already use.

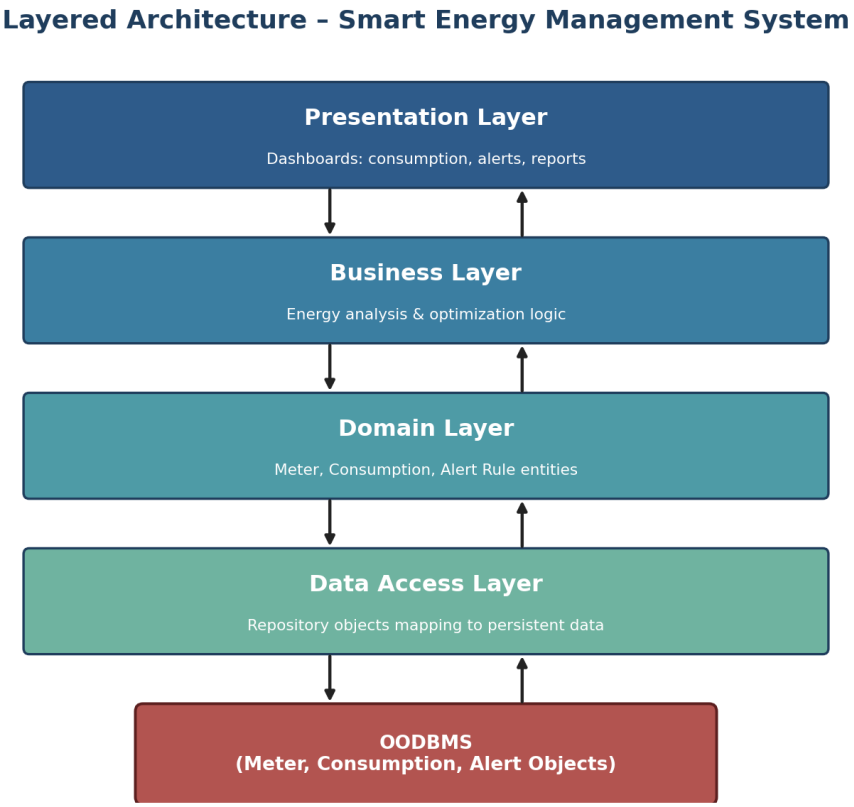

Figure 5.1 — Layered architecture and OODBMS interaction for the Smart Energy Management System.

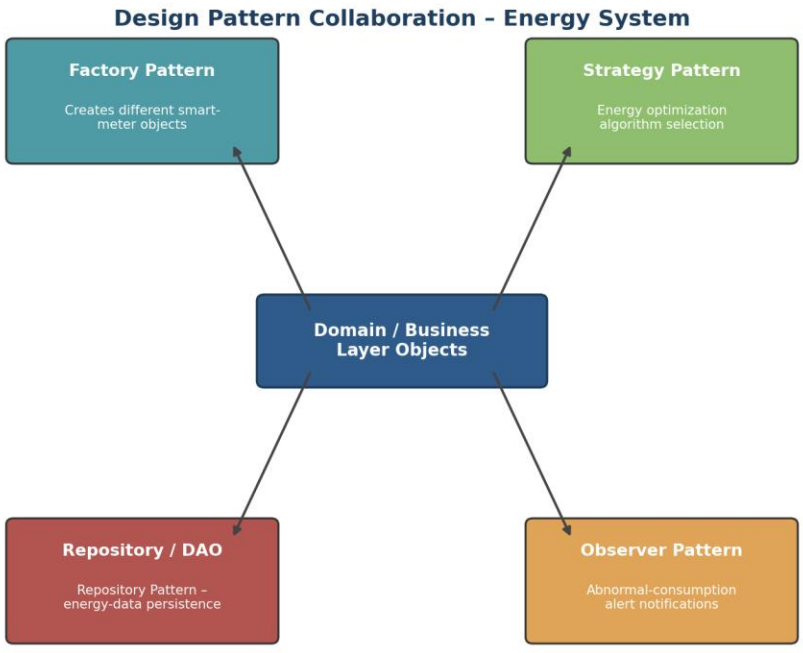

Figure 5.2 — Collaboration between the Factory, Strategy, Observer and Repository/DAO patterns and the domain layer.

(b) Layer Communication with the OODBMS

Meter readings flow from the presentation layer's dashboards down to the business layer, which performs energy analysis using domain objects (SmartMeter, Consumption, AlertRule). When a reading or computed value must be stored or looked up, the domain/business layer invokes the Repository Pattern's interface rather than accessing the database directly. The repository persists SmartMeter, Consumption and Alert objects natively in the OODBMS, keeping their relationships intact — for example a SmartMeter object holding a direct reference to its historical Consumption objects — which supports fast time-series analysis without relational joins. When the dashboard requests a consumption trend or triggers an alert investigation, the repository retrieves the relevant persistent objects from the OODBMS and the business layer transforms them into the charts and alerts shown at the presentation layer. Because only the data-access layer touches the OODBMS, the energy company can scale the database horizontally to accommodate growing numbers of smart meters, or extend it to store renewable-generation data, without changing the analysis logic or the operator dashboards.